# Property Peace Stripe Payment Incident Remediation Report

| |
|---|
| **Document version:** 1.0 |
| **Prepared:** July 30, 2026 |
| **System:** Property Peace |
| **Classification:** Internal / Confidential |
| **Technical change:** c0f182dbdc61e84c51fc98b6792a0f9839c843a7 (feat(payments): secure rent settlement and transfers) |
| **Status:** Remediation deployed to the dev branch; payment and transfer gates remain disabled |

## 1. Executive summary

Property Peace identified a material financial-control weakness in its prior Stripe Connect rent-payment design. Rent proceeds could be transferred to a connected landlord account before the underlying payment was sufficiently settled. If the payment was later returned, disputed, or charged back, Stripe could debit the Property Peace platform balance after funds had already moved to the connected account. This created direct platform exposure and contributed to the negative-balance event that initiated the review.

The rent-payment flow was redesigned around server-authoritative payment creation, separate delayed transfers, webhook-authoritative settlement, durable accounting, exact loss tracking, and idempotent transfer recovery. The redesign was pushed to the dev branch in commit c0f182d. The development database migration was applied and validated against SQL Server. Payment intake and landlord transfer release remain independently disabled until the remaining rollout prerequisites are completed.

## 2. What happened and why it mattered

### Observed risk

- Customer payments could succeed and funds could be moved to connected accounts before return and dispute risk had sufficiently elapsed.
- A later chargeback or ACH return could debit the platform while the connected account retained the earlier transfer.
- Local accounting did not previously provide the complete durable state needed to reconcile every payment, transfer, refund, dispute, reversal, and ambiguous external response.
- Financial correctness depended too heavily on client-supplied or browser-confirmed data rather than a server-owned obligation and Stripe webhook evidence.

### Business impact

The design exposed Property Peace to negative Stripe balances, recovery effort, inaccurate rent balances, duplicate or excessive reversals, and incomplete audit evidence. The remediation therefore addressed both application security and business financial controls.

# 3. Changes implemented

## 3.1 Immediate containment and fail-closed operation

- Added independent Stripe:RentPaymentsEnabled and Stripe:TransfersEnabled gates.
- An absent or false gate prevents the corresponding money movement.
- Both gates remain false; no payment or transfer capability was enabled by this change.
- Kept webhook processing available so existing Stripe events can still be reconciled while new money movement is closed.

## 3.2 Server-authoritative payment creation

- The API now derives and validates the lease, organization, tenant, amount, currency, and destination account on the server.
- Tenant membership is revalidated inside the protected allocation boundary.
- Browser confirmation is treated as acknowledgment only and cannot create final accounting records.
- Stripe PaymentIntent metadata and charge details must match the durable local payment aggregate.

## 3.3 Separate charges and delayed landlord transfers

- Replaced immediate destination-style money movement with a platform charge followed by a separate transfer.
- A successful charge is held before landlord transfer eligibility.
- The implemented conservative minimum holds are seven elapsed days for card payments and fourteen elapsed days for ACH/bank payments.
- Immediately before a transfer, the service revalidates the authoritative charge identity, source PaymentIntent, amount, currency, payment state, and destination.

## 3.4 Durable payment and transfer state machine

A new financial.StripeRentPayments aggregate records the payment lifecycle, including:

- operation, PaymentIntent, charge, lease, organization, tenant, currency, and destination identifiers;
- settlement, hold, transfer eligibility, transfer, retry, and reconciliation timestamps;
- refund, dispute, risk, transfer-reversal, retry, and failure information;
- optimistic concurrency through SQL Server rowversion;
- unique indexes for operation ID, PaymentIntent ID, and Stripe transfer ID.

The state machine explicitly represents held, transfer-pending, reconciliation-pending, transferred, reversal-pending, reversed, blocked, and recovery-failed outcomes.

## 3.5 Atomic rent allocation and ledger synchronization

- Rent allocation uses a serializable relational transaction.
- SQL Server sp_getapplock serializes the obligation reservation across application instances.
- Tenant-to-lease membership is revalidated inside the lock and transaction.
- The PaymentIntent, amount, currency, lease, organization, tenant, operation ID, and charge must match the durable aggregate.
- Allocation cannot be marked complete unless the required general-ledger payment entry was created in the same transaction.
- Webhook replay cannot duplicate completed allocation.

### 3.6 Webhook-authoritative settlement

- Signed Stripe webhook events are the authority for final settlement and problem-state transitions.
- Event processing is replay-safe and preserves durable webhook tracking.
- The Stripe event creation timestamp is used as the authoritative settlement timestamp.
- Sensitive request headers, including authorization, cookies, and Stripe signatures, are not enumerated or logged.

### 3.7 Exact refund, dispute, and loss accounting

- The system records exact cumulative refunded and disputed amounts.
- Partial refunds reopen only the exact lost portion of the tenant's rent obligation.
- Repeated events update durable loss state rather than duplicating accounting.
- Required reversing ledger entries are written atomically with loss accounting.
- Database constraints prevent negative counters or aggregate losses above the original payment.

### 3.8 Incremental transfer reversals and ambiguous-outcome reconciliation

- If $30 of a $100 transferred payment is refunded, the target recovery is $30—not the full $100.
- If cumulative loss later increases from $30 to $50, the next reversal is only the additional $20.
- The exact in-flight reversal target and increment are persisted before Stripe is called.
- Deterministic idempotency keys ensure that a lost or ambiguous Stripe response replays the same operation rather than creating a duplicate.
- A later refund cannot overwrite or bypass an earlier unresolved reversal.

### 3.9 Database integrity controls

The migration adds constraints requiring:

- a positive original amount;
- nonnegative refund, dispute, and reversal counters;
- cumulative refunds and disputes not to exceed the original payment;
- reversal targets and increments to remain internally consistent and within the original amount.

## 4. Verification completed

The final implementation review recorded the following evidence:

- API tests: **225 passed, 0 failed**.
- API build: passed.
- Isolated web production build: passed.
- Migration SQL generation: passed (115 lines).
- CRLF-aware diff validation: passed.
- Changed-file security scan found zero live Stripe secrets, private keys, enabled payment gates, destination-charge APIs, or sensitive-header enumeration.
- Development SQL Server migration 20260730012758_AddSeparateChargesDelayedRentTransfers was applied.

- Real SQL Server checks validated the four financial constraints, rowversion stale-update rejection, serializable sp_getapplock contention/release, and exactly-once concurrent allocation with one payment, one ledger entry, and one completion marker.
- Independent review result: **PASS WITH ROLLOUT PREREQUISITE**, with no remaining code-level High or Critical flaw identified in the redesigned flow.

# 5. Current status and remaining prerequisites

## Completed

- Redesign committed and pushed to origin/dev.
- Development migration applied and verified.
- Payment and transfer feature gates remain disabled.
- Production was not changed by commit c0f182d.

## Required before production enablement

1. Remove plaintext development credentials from tracked configuration and rotate affected credentials.
2. Remediate or formally risk-accept vulnerable .NET and frontend dependencies; the last review reported vulnerable AutoMapper/transitive .NET packages and 26 frontend audit findings (2 critical, 17 high, 4 moderate, 3 low).
3. Configure dedicated Stripe test-mode keys, matching webhook secret, and test connected accounts in the deployed development environment without exposing secret values.
4. Complete end-to-end development acceptance for replay, tampering, delayed card/ACH settlement, refunds, disputes, ambiguous transfer/reversal outcomes, and cleanup.
5. Confirm operational alerting and ownership for long-processing payments, failed transfers, reconciliation-pending records, reversals, disputes, and recovery failures.
6. Approve a staged rollout and rollback plan. Enable payment intake separately from transfer release and observe results before expanding access.

# 6. Policy impact

Before this report, the repository contained a security assessment and an implementation summary, but not a controlled, owner-approved information-security policy. Those documents describe findings and technical changes; they do not establish governance, ownership, approval, review frequency, exceptions, incident response, payment controls, or business-continuity requirements.

A new draft **Information Security and Payment Operations Policy** accompanies this report. It converts the lessons from this incident into mandatory operating rules. The policy is not considered adopted until the business owner assigns accountable roles, approves it, records an effective date, and communicates it to everyone with system access.

# 7. Management decisions requested

- Approve the accompanying policy and assign the Security Owner and Payment Operations Owner.
- Confirm the vulnerability-remediation and credential-rotation owners and deadlines.
- Keep both Stripe feature gates disabled until all production prerequisites have written evidence and sign-off.
- Require a post-rollout review and at least annual policy review thereafter.

# 8. Evidence references

- Git commit: c0f182dbdc61e84c51fc98b6792a0f9839c843a7
- Migration: property-peace-api/Migrations/20260730012758_AddSeparateChargesDelayedRentTransfers.cs
- Payment state: property-peace-api/Models/StripeRentPayment.cs
- Allocation: property-peace-api/Services/StripeRentPayments/StripeRentAllocationService.cs
- Payment/transfer/recovery processing: property-peace-api/Services/StripeRentPayments/StripeRentPaymentService.cs
- Loss accounting: property-peace-api/Services/StripeRentPayments/StripeRentLossAccountingService.cs
- Stripe webhooks: property-peace-api/Services/StripeService/StripeWebhookService.cs
- Regression suites: property-peace-api.Tests/Services/StripeRentPayments/

---

**Document limitation:** This report records the implemented technical and operational remediation. It is not legal advice, an audit opinion, a PCI compliance certification, or confirmation that the production feature is ready to enable.